Beginner Guide To Secure Environment Variables And Secrets

Hardcoding passwords and API keys into application code is the easiest way to cause a major security breach. Learning to manage config variables separately makes your app secure, portable, and ready for production.

What you'll learn:

1. Setting Shell Environment Variables
2. Blocking Secrets from Git Tracking
3. Injecting Config into Docker Containers
4. Managing Kubernetes Cluster Secrets

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Setting Shell Environment Variables

Environment variables are dynamic values loaded by your operating system at runtime. Instead of changing your code when switching from local testing to production, you update variables outside the application. In Linux and macOS terminals, you set these using the export command.

```
$export PORT=8080 && export DB_HOST="localhost"
```

Environment Variables & Secrets: Manage Config Safely

Wednesday, August 26, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Blocking Secrets from Git Tracking

The single most important rule in modern software development is never pushing private keys to Git repositories. A .gitignore file acts as a filter, preventing sensitive local files like .env from being uploaded to public or private repos.

```
$echo ".env" >> .gitignore && git status
```

Environment Variables & Secrets: Manage Config Safely

Wednesday, August 26, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Injecting Config into Docker Containers

Containers require external configuration to run across different environments seamlessly. You can pass environment variables to a Docker container using the -e flag or by supplying an entire .env file using the --env-file option at runtime.

```
$docker run -d --env-file .env -p 3000:3000 my-node-app
```

Environment Variables & Secrets: Manage Config Safely

Wednesday, August 26, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Managing Kubernetes Cluster Secrets

In Kubernetes, sensitive configuration like passwords, SSH keys, or access tokens should never be stored in plain text YAML files. Kubernetes Secrets allow you to store confidential data encrypted within your cluster and pass them to pods safely.

```
$kubectl create secret generic api-keys --from-literal=API_KEY=xyz123
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Never hardcode passwords, secret keys, or database URIs directly in your source code.
+ Always add your .env file to .gitignore before committing any code to Git.
+ Maintain a template file named .env.example with dummy values for developer setup.
+ Use consistent naming conventions like DATABASE_URL, PORT, and APP_ENV across all tools.
+ Use managed tools like AWS Secrets Manager or HashiCorp Vault for production deployments.

Watch Out For

! Accidentally committing secret keys into public GitHub repositories.
! Printing full environment variables to server console logs during debugging.
! Assuming client-side frontend code can safely hold secret API keys.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh